1.

```
import pandas as pd
df = pd.read_csv("enjoysport.csv")
print("The dataset is:\n", df)
data = df.iloc[0:, 0:7].values
h = ['0', '0', '0', '0', '0', '0']
def Generalize(h, a):
    for i in range(len(a)):
        if h[i] != a[i]:
            h[i] = '?'
        else:
            h[i] = a[i]
    return h
X = data[:, 0:6]
Y = data[:, -1]
if Y[0] == "Y":
    h = X[0, :]
else:
    print("Error: First Example is not positive")
for i in range(len(X)):
    if Y[i] == "Y":
        h = Generalize(h, X[i, :])
print("The more general than hypothesis is:", h)
```

2.

```
import numpy as np
concepts = np.array([
    ['sunny','warm','normal','strong','warm','same'],
    ['sunny','warm','high','strong','warm','same'],
    ['rainy','cold','high','strong','warm','change'],
    ['sunny','warm','high','strong','cool','change']
])
target = np.array(['yes','yes','no','yes'])
def candidate_elimination(concepts, target):
    S = concepts[0].copy()
    G = [["?" for _ in range(len(S))]]
    for i, h in enumerate(concepts):
        if target[i] == "yes":
            for x in range(len(S)):
                if h[x] != S[x]:
                    S[x] = "?"
            G = [g for g in G if all(g[x] == "?" or g[x] == h[x] or
S[x] == "?" for x in range(len(S)))]
        else:
            new_G = []
            for x in range(len(S)):
                if S[x] != "?" and h[x] != S[x]:
```

2.

```
                g = ["?"] * len(S)
                g[x] = S[x]
                new_G.append(g)
        G.extend(new_G)
    return S, G
S_final, G_final = candidate_elimination(concepts, target)
print("Final Specific Hypothesis:", S_final)
print("Final General Hypotheses:", G_final)
```

3.

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import LabelEncoder
data = pd.DataFrame({
    'Outlook':
['Sunny','Sunny','Overcast','Rain','Rain','Rain','Overcast','Sunn
y','Sunny','Rain','Sunny','Overcast','Overcast','Rain'],
    'Temperature':
['Hot','Hot','Hot','Mild','Cool','Cool','Mild','Hot','Cool','Mild','
Mild','Mild','Hot','Mild'],
    'Humidity':
['High','High','High','High','Normal','Normal','Normal','High','
Normal','Normal','Normal','High','Normal','High'],
    'Wind':
['Weak','Strong','Weak','Weak','Weak','Strong','Strong','Weak','
Weak','Weak','Strong','Strong','Weak','Strong']
    'PlayTennis':
['No','No','Yes','Yes','Yes','No','Yes','No','Yes','Yes','Yes','Yes','
Yes','No']
})
le = LabelEncoder()
encoded_data = data.copy()
for col in encoded_data.columns:
    encoded_data[col] = le.fit_transform(encoded_data[col])
```

3.

```
X = encoded_data.drop('PlayTennis', axis=1)
y = encoded_data['PlayTennis']
model = DecisionTreeClassifier(criterion='entropy')
model.fit(X, y)
sample = pd.DataFrame({
    'Outlook': ['Sunny'],
    'Temperature': ['Cool'],
    'Humidity': ['High'],
    'Wind': ['Strong']
})
for col in sample.columns:
    sample[col] = le.fit_transform(sample[col])
prediction = model.predict(sample)
print("Prediction (0=No, 1=Yes):", prediction[0])
```

4.

```
import numpy as np
def sigmoid(x):
    return 1 / (1 + np.exp(-x))
def sigmoid_derivative(x):
    return x * (1 - x)
X = np.array([[0,0],
              [0,1],
              [1,0],
              [1,1]])
y = np.array([[0],
              [1],
              [1],
              [0]])
np.random.seed(1)
input_neurons = 2
hidden_neurons = 2
output_neurons = 1
W1 = np.random.uniform(size=(input_neurons,
hidden_neurons))
W2 = np.random.uniform(size=(hidden_neurons,
output_neurons))
b1 = np.random.uniform(size=(1, hidden_neurons))
```

4.

```
b2 = np.random.uniform(size=(1, output_neurons))
learning_rate = 0.5
epochs = 10000
for epoch in range(epochs):
    hidden_input = np.dot(X, W1) + b1
    hidden_output = sigmoid(hidden_input)
    final_input = np.dot(hidden_output, W2) + b2
    predicted_output = sigmoid(final_input)
    error = y - predicted_output
    d_output = error * sigmoid_derivative(predicted_output)
    error_hidden = d_output.dot(W2.T)
    d_hidden = error_hidden *
sigmoid_derivative(hidden_output)
    W2 += hidden_output.T.dot(d_output) * learning_rate
    W1 += X.T.dot(d_hidden) * learning_rate
    b2 += np.sum(d_output, axis=0, keepdims=True) *
learning_rate
    b1 += np.sum(d_hidden, axis=0, keepdims=True) *
learning_rate
print("Predicted Output after Training:\n", predicted_output)
```

5.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score,
classification_report
data = pd.read_csv("data.csv")
X = data.iloc[:, :-1]
y = data.iloc[:, -1]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
model = GaussianNB()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
print("\nClassification Report:\n", classification_report(y_test,
y_pred))
```